

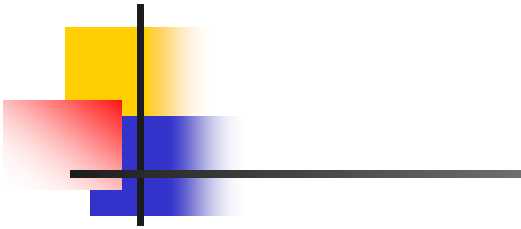
第四章

80C51单片微机的程序设计

4.1 概述

- 汇编语言格式：
  - **计算机语言的种类：**

    机器语言， 汇编语言， 高级语言



- **机器语言**

用二进制代码表示的指令称为机器语言系统，用机器语言构成的程序称为目的（目标）程序。

特点：

- **可以被计算机识别并执行**
- **记忆困难**
- **难以阅读，不易看懂**

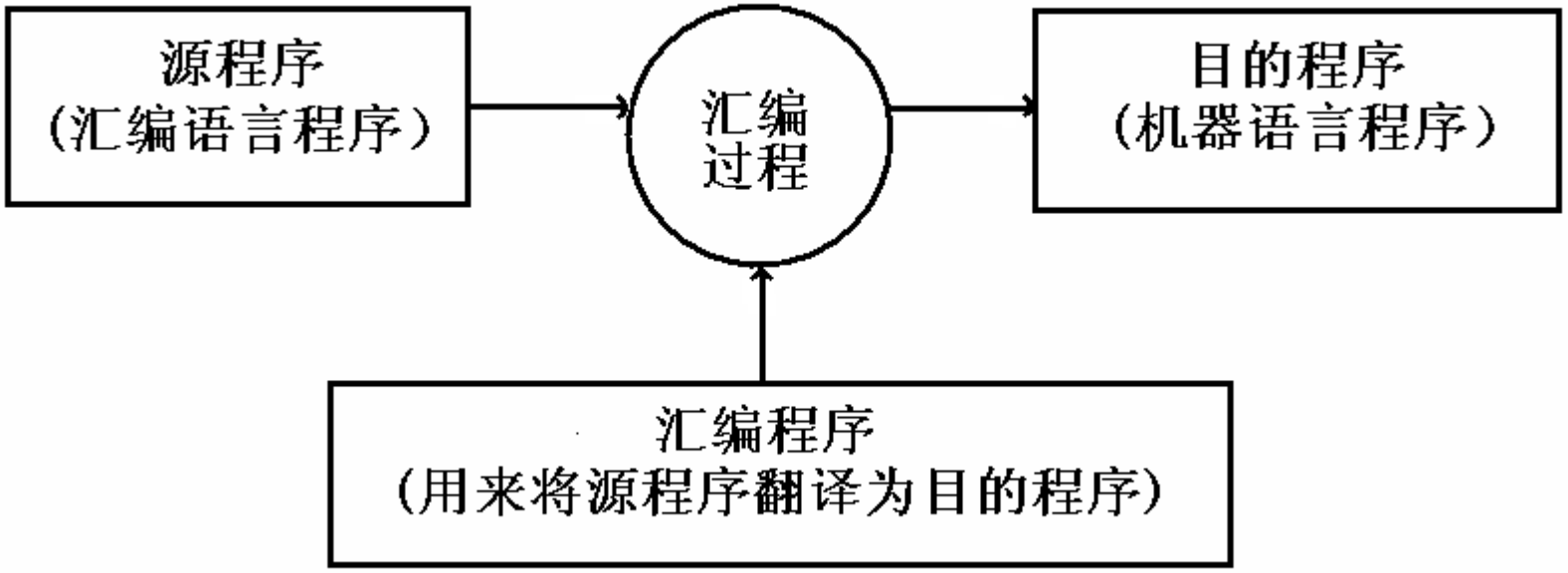
- **汇编语言**

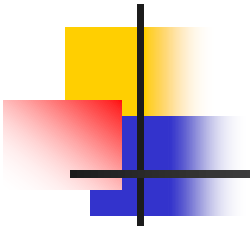
是用英文符号（助记符）表示的指令，用汇编语言构成的程序称为汇编语言源程序。

特点：

- **机器语言和汇编语言都是面向机器的**
- **与单片机硬件系统相关，移植困难。**
- **程序效率高，占用存储空间小，运行速度快**
- **直接管理和控制硬件设备**

汇编语言程序、汇编程序与机器语言程序关系图



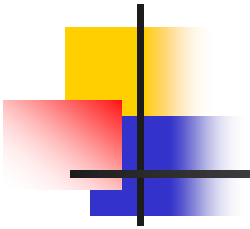


- **高级语言**

参照数学语言而设计的、近似于人们日常用语的语言，面向过程的。如C-51、C 、MBASIC、PL/M

特点：

- **编程容易，移植方便**
- **占用的内存较多。**
- **要求单片机开发系统带相应的编译软件**



- **汇编语言语句的种类：**

指令语句

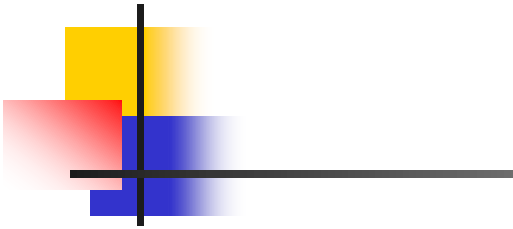
每一个指令对应一个目标代码

伪指令语句

机器汇编时，在源程序中应向编译软件发出如何完成汇编的指示信息。这些指示信息就是伪指令，没有**目标代码**

宏指令语句（MACRO)

用以代替汇编语言源程序中重复使用的程序段



- **汇编语言语句的格式：**

指令语句格式：

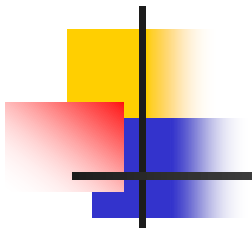
[标号(名字)]: 助记符(操作码) [操作数(参数)]； [注释]

[ ]内为可选部分，视需要而定

1）标号　语句地址的标志符号

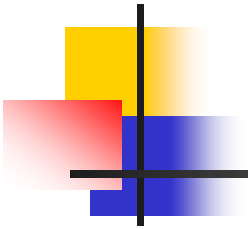
如：

```
LOOP:        MOV   A,      #0F0H          ; 赋值
```



注意事项：

- 标号由1 ~ 8个ASCII字符组成，第一个字符必须是字母。
- **不能使用本汇编语言已经定义了的符号作为标号**，如指令助记符、伪指令以及寄存器的名称等。
- **标号后面必须跟冒号。**
- **同一标号在一个程序中只能定义一次**，不能重复定义。
- 没有其它语句访问时，标号可以省略。



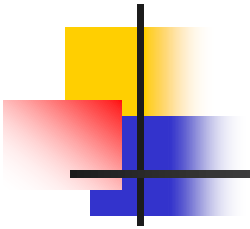
2）操作码　规定语句执行的操作

是汇编指令中唯一不能空缺的

3）操作数　给指令的操作提供数据或数据所在的地址

- 操作数的数量0~2个。多于一个时，操作数之间用逗号分隔。
- 操作码与操作数之间用空格分开
- 十六进制数需要加后缀”H”, 如果以A~F开头，通常需要在前面加0，防止编译出错。

```
LOOP:        MOV   A,      #0F0H          ; 赋值
```



4）注释　不属于语句的功能部分，便于阅读程序。

以分号开始，长度不限，另起一行时也必须以分号开始。

5）分隔符

•冒号 （：）用于标号后

•空格 （　　）　用于操作码和操作数之间

•逗号 （，）　用于操作数之间

•分号 （；）用于注释前

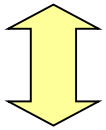


例：把片外存储器2200H单元中的数送入片内70H单元中。

```
    标号域  操作码域    操作数域              注释域
BEGIN：MOV      DPTR, #2200H   ；(DPTR)=2200H
       MOV      R0, #70H       ；(R0)=70H
       MOVX     A, @DPTR       ；(A)=((DPTR))
       MOV      @R0, A         ；((R0))=(A)
```

- **伪指令语句的格式：**

  名字　定义符　参数　；[注释]



[标号(名字)]: 助记符(操作码) [操作数(参数)]；[注释]

- **某些伪指令在其定义符可以没有名字**
- 伪指令语句与指令语句主要不同是在其名字后面没有冒号
- 参数多于1个，用逗号隔开
- 注释以分号开始，长度不限，与指令语句一样

伪指令语句：

汇编语言程序变成单片机能执行的机器语言的过程称为汇编，可以手工汇编，也可以由计算机汇编

机器汇编时，在源程序中应向编译软件发出如何完成汇编的指示信息。**这些指示信息就是伪指令**。

特点：

- 不产生机器指令代码
- 仅在汇编的时候使用，不同编译软件的伪指令有所不同



1. ORG (ORiGin)：

汇编起始地址伪指令

[<标号>:]　　ORG　　<地址16位>

- 通常在由ORG定位时，其地址应当由小到大，不能重叠。
- 有效范围一直到下一条ORG伪指令出现为止。
- 跟在ORG伪指令后面的程序段或数据段是绝对地址还是浮动地址段，依赖于ORG右边的表达式性质。

例：

绝对地址

ORG 1000H

START：MOV A，＃12H

浮动地址

ORG RELOCA

SUBROU：……

2. END (END of assembly)

汇编终止命令

[<标号>:]　　END　　[<表达式>]

3. EQU (EQUate)

赋值命令

<字符名称>　EQU　　<赋值项>

```
COUNT   EQU     30H
MOV     COUNT，#25H   ；立即数25H传送至30H单元
```

4. DL定义标号值伪指令

<标号>　DL　<表达式>

例:　COUNT DL 3000H　；S定义标号COUNT的值为
　　；3000H

　　COUNT DL COUNT十1　；重新定义COUNT的值为
　　；3000H＋l

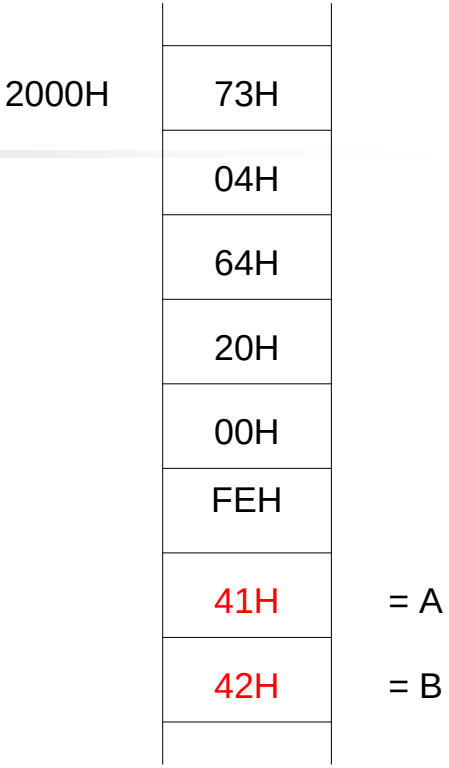
DL和 EQU比较:

- 将表达式值赋予标号,
- DL语句可更改已定义的标号值;
- 用EQU语句定义的标号，在整个源程序中不能更改。

5. DB (Define Byte)

定义数据字节命令

[<标号>:]　DB　<8位数表 >



如:

```
        ORG 2000H
TABLE:  DB   73H，04H，100，32，00，  - 2
        DB   “ABC”
```

查表时，为确定数据区的起始地址可采用：

1）根据DB命令前一条指令的地址确定

```
8100H  7449    MOV          A，#49H
8102H   TAB：DB    0C0H，0F9H，0A4H，0B0H
                  DB  99H，… …
```

2）用ORG命令

```
                ORG          8200H
TAB：           DB    0C0H，0F9H，0A4H，0B0H
                DB    99H，… …
```

6. DW (Define Word)

定义数据字命令

[<标号>:]　DW　<16位数表 >

ORG　8100H

DW　100H，3456H，814

8100H 01H　；高位低地址

8101H 00H

8102H 34H



7. DS (Define Storage)

定义存储区命令

[<标号>:]　　DS　　<8位或16位数 >

如：　BASE：DS　　100H

；从标号代表的地址开始保留100H个单元，备用

对于MCS-51，DB、DW、DS只能对程序存储器使用，不能对数据存储器使用

8. BIT 位定义命令

<字符名称> BIT <位地址 >

ABC BIT P3.1

；在程序中ABC就代表位地址P3.1

例1. 伪指令应用

```
ORG  8100H
BUFFER  DS  10H
DW  “A B”
DW  100H，1ACH，-814
```

说明：① 从8100H至810FH为缓冲区空间

② (8110H)=41H (‘A’)

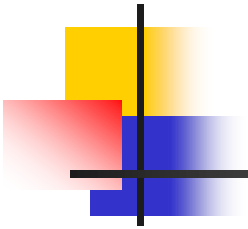
(8111H)=42H (‘B’)

③ 8112H单元起存放01H、00H、01H、ACH、FCH、D2H

4.2 80C51汇编语言程序设计

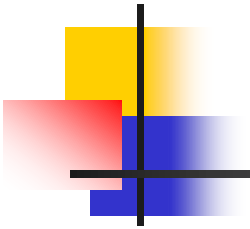
- 程序设计的方法、步骤、特点：
  - **分析问题：确定需要做什么?**
  - **求解问题：确定如何做？如算法，流程**
  - **实现：用相应的指令组来实现**
  - **调试：**

对复杂程序，通常需要流程图来规范及具体化程序设计



➢ 汇编语言程序设计的特点

- 数据的存放、寄存器和工作单元的使用等要由设计者安排。
- 要求设计人员对所使用计算机的硬件结构有较详细的了解，特别是对各类寄存器、端口、定时器/计数器、中断等内容能熟练掌握，以便在程序设计中使用。
- 程序设计的技巧性较高，需要软件与硬件的结合



➢ 汇编语言程序的基本结构

- 顺序结构
- 分支结构
- 循环结构
- 子程序调用
- 中断服务子程序



4.2.1 顺序结构：

程序执行时一条一条按顺序执行指令，直到结束

特点：

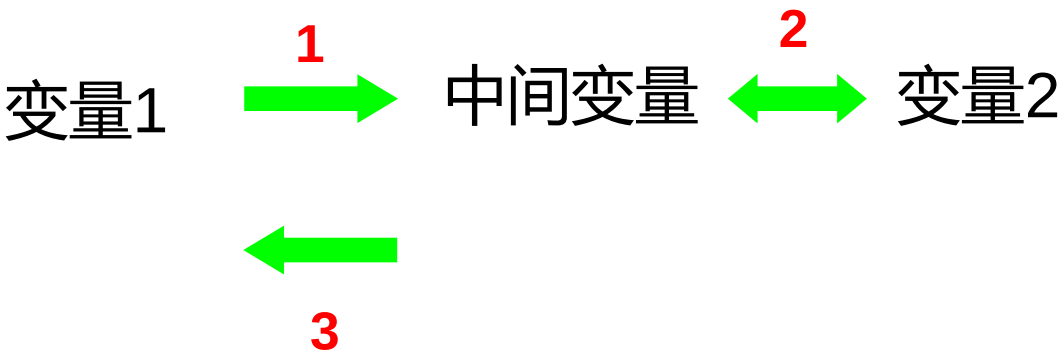
- **最基本、最简单的结构**
- **使用最广泛**
- **组成复杂程序的基础**

例： 数据传送和交换。

将R0与R7内容互换，R4与内存20H单元内容互换。

分析问题：实现互换

解决问题：通过一个中间变量实现互换



实现： XCH, MOV

例4.2.1-1

```
        ORG 0000H
        LJMP XCHR
        ORG 0030H
XCHR:   MOV    A，R0  ;A ← (R0)
        XCH    A，R7  ;A ↔ (R7)，A与R7互换
        XCH    A，R0  ; R0与A内容互换，实现R0 ↔ R7
        (MOV   R0, A   )
        MOV    A，R4  ;
        XCH    A，20H;
        XCH    A，R4   ; R4与20H单元内容互换
```

例4.2.1-2

例： 不带符号多字节加法。

设被加数存放于片内RAM的20H（低位字节）、21H（高位字节），加数存放于22H（低位字节）和23H（高位字节），运算结果的和数存放于20H（低位字节）和21H（高位字节）中。实现16位相加。

	21H	20H
+	23H	22H
	21H	20H

低字节相加，进位必须考虑

高字节相加，进位保留在**C**中

```
        ORG 0000H
        SJMP STRAT
        ORG 0030H
START:  MOV   R0，＃20H      ；将20H地址送R0
        MOV   R1，＃22H      ；将22H地址值送R1
        MOV   A，@R0         ；被加数低字节内容送A, A=(20H)
        ADD   A，@R1         ；低字节数相加, A = (A) + (22H)
        MOV   @R0，A         ；低字节数和存20H中
        INC     R0           ；指向被加数高位字节 R0 = 21H
        INC     R1           ；指向加数高位字节 R1=23H
        MOV   A，@R0         ；被加数高位字节送A
        ADDC  A，@R1         ；高字节数带进位相加
        MOV  @R0，A          ；高字节数和存21H中
```

```
        CLR     A
        ADDC    A,  #00H
        MOV     10H, A          ；进位暂存于10H中
HERE:   SJMP    HERE
```

如果相加结果高字节的最高位产生进位且有意义时，应对标志CY位检测并处理之。

例4.2.1-3

例: 带符号双字节二进制数加减法程序

已知二个16位的带符号数分别存放在工作寄存器R2(高字节) 、R3(低字节) 和R4(高字节)、R5(低字节) 中，二个数的D15位为符号位。请编写多字节带符号数的加法和减法运算程序。减法入口地址为标号BSUB，加法入口地址为BADD。运算结果存入R6(高字节)和R7(低字节) 。

(有符号数为非补码表示，符号位＋原码 表示)

+4096 = 1000H = **0**001 0000 0000 0000 B　正数

-4096 = 9000H = **1**001 0000 0000 0000 B　负数



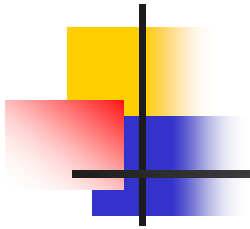
问题分析：

实现加减法：加减法可以统一

分别写加法、减法程序，或者通过将减法程序中的减数符号位取反，**用加法来实现减法，即** A – B = A + (-B)

加法如何实现：

- A和B符号相同（2个正数或负数）：加法，
- A和B符号相异（A正B负或A负B正）：减法
- 加法保留被加数符号位
- 减法符号位分成下列2中情况:



数值A≥数值B：相减的差就是结果，保留被减数的符号位

如：+9+(-6) = +(9-6)=+3　　- 9 + 6 = - (9-6) = - 3

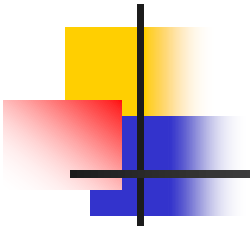
数值A<数值B：相减的差就是结果的补码，需要调整

结果取补码，被减数的符号为取反

如：+6+(-9) = +(6-9)= - 3　　- 6 + 9 = - (6-9) = + 3

	0000 0110	
数值相减	− 0000 1001	
	C=1 1111 1101	
	取补 0000 0011	
	符号位取反 1000 0011	

	0000 0110	
	− 0000 1001	
	C=1 1111 1101	
	取补 0000 0011	
	符号位取反 0000 0011	



求解问题：

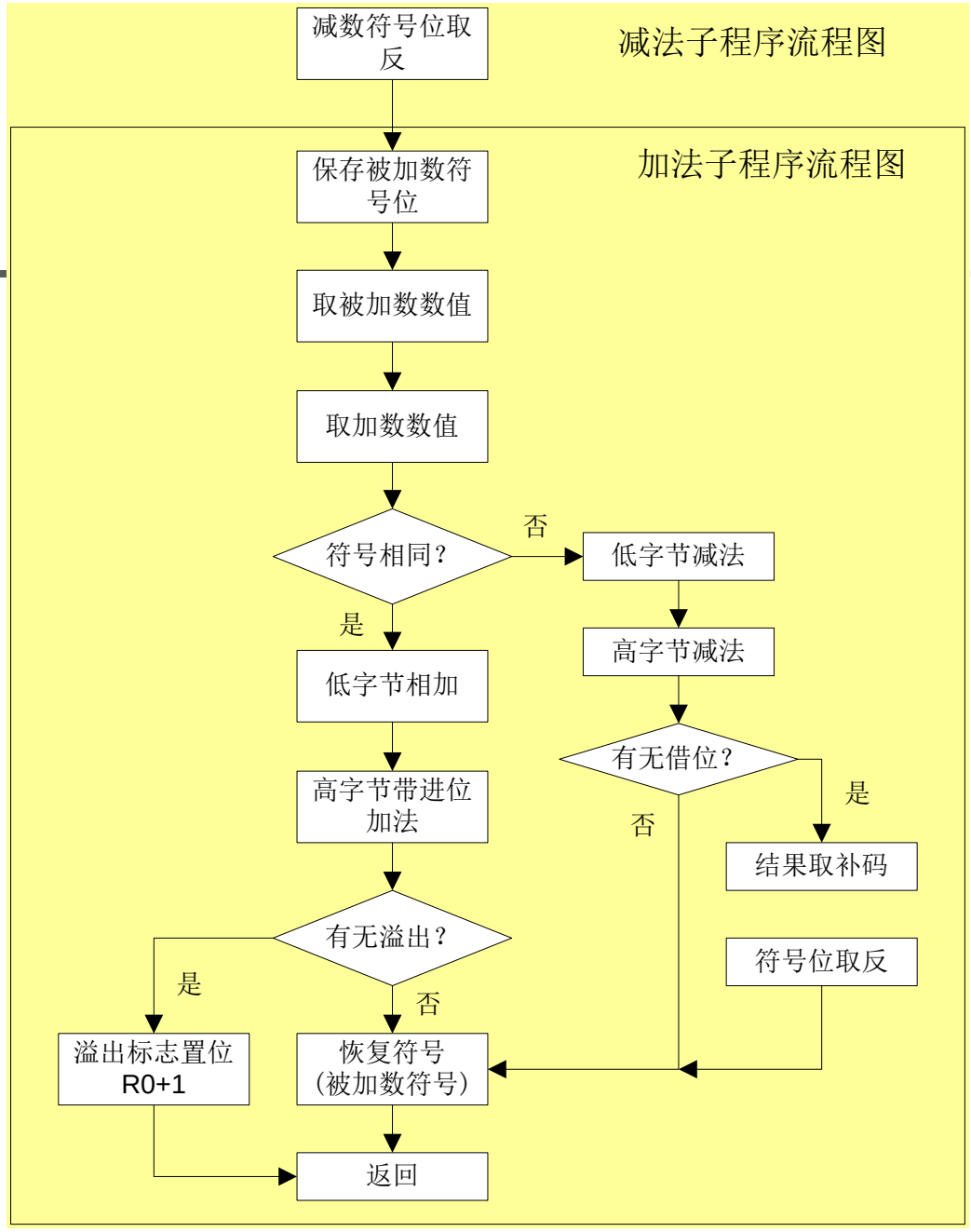
1) 编写减法程序只需要将减数的符号位取反，统一为加法

2) 比较2个数为同号或者异号，保存被加数的符号位

3) 同号，数值部分相加，结果的符号为保留被加数符号

4) 异号，数值部分相减，相减为正，符号保留被加数符号；相减为负，结果取补，符号取反

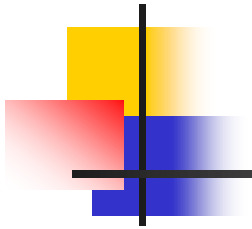
5) 其他：相加存在溢出的可能(>0FFFFH)；


减法子程序流程图
减数符号位取反
加法子程序流程图
保存被加数符号位
取被加数数值
取加数数值
符号相同？
否
低字节减法
是
高字节减法
低字节相加
有无借位？
高字节带进位加法
是
否
结果取补码
有无溢出？
符号位取反
是
否
溢出标志置位 R0+1
恢复符号（被加数符号）
返回

；两字节带符号数加减运算，DATA1-DATA2，　主程序

；DATA1+DATA2，溢出R0为1

```
DATA1H    EQU   32H
DATA1L    EQU   5DH
DATA2H    EQU   87H
DATA2L    EQU   46H
          ORG   0000H          ；  主程序
          MOV   SP，#60H       ；  设置堆栈指针
          MOV   R0，#00H       ；溢出标志初值
          MOV   R2，#DATA1H        ；送被减数
          MOV   R3，#DATA1L
          MOV   R4，#DATA2H        ；送减数
          MOV   R5，#DATA2L
          LCALL     BSUB           ；调用减法子程序
```

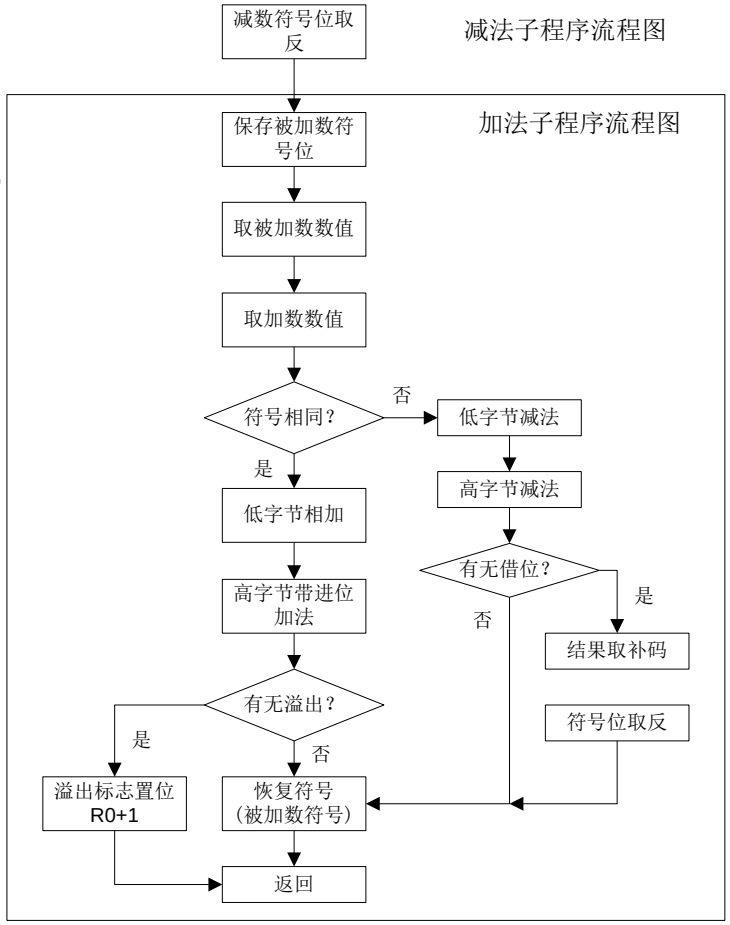


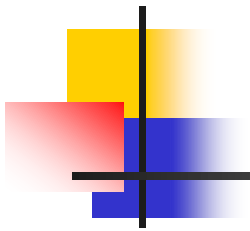
```
MOV     R0，#00H     ；溢出标志初值
MOV     R2，#DATA1H；送被加数
MOV     R3，#DATA1L
MOV     R4，#DATA2H ；送加数
MOV     R5，#DATA2L
LCALL   BADD     ；调用加法子程序
SJMP    $
```



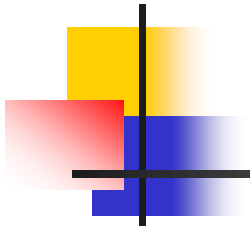
```
BSUB:   MOV   A，R4        ；减数符号取反
        CPL   ACC.7
        MOV   R4，A
        ;符号位取反后，减法就可以调用加法程序
BADD:   MOV   A，R2        ；加法，取被加数符号
        MOV   C，ACC.7     ；被加数符号位送F0保存
        MOV   F0，C
        XRL   A，R4        ；两数异或（符号是否同号）
        MOV   C，ACC.7     ；同号C=0，异号C=1
```

```
MOV   A，R2 ；取被加数去符号位
CLR   ACC.7
MOV   R2，A
MOV   A，R4 ；取加数去符号位
CLR   ACC.7
MOV   R4，A
JC    JIAN   ；两数异号转JIAN
```



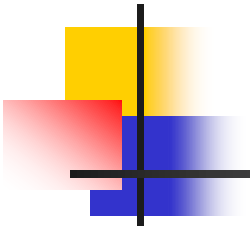


```
JIA:   MOV     A，R3          ；两数同号，低字节加
       ADD     A，R5
       MOV     R7，A          ；存低字节和
       MOV     A，R2          ；高字节带进位加
       ADDC    A，R4
       MOV     R6，A          ；存高字节和
       JB      ACC.7，QAZ；符号位为1转溢出处理
QWE:   MOV     C，F0          ；结果符号处理
       MOV     ACC.7，C
       MOV     R6，A
       RET
```



```
JIAN：MOV     A，R3        ；取被减数低字节
      CLR     C            ；清进位
      SUBB    A，R5        ；低字节相减
      MOV     R7，A        ；存低字节差
      MOV     A，R2        ；取被减数高字节
      SUBB    A，R4        ；高字节相减
      MOV     R6，A        ；存高字节差
      JNB     ACC.7，QWE；判差的符号，为0则转移
BMP：MOV      A，R7        ；差为负，低字节取补
      CPL     A
```

```
        ADD     A，#1
        MOV     R7，A
        MOV     A，R6          ；高位取补
        CPL     A
        ADDC    A，#0
        MOV     R6，A
        CPL     F0             ；保存在F0中的符号取反
        SJMP    QWE            ；转结果符号处理
QAZ：   INC     R0             ；溢出标志处理
        RET
        END                    ；
```



例4.2.1-4

例：双字节乘法。

多字节乘法的基础是加法。分别相乘后对应字节相加(个位、十位、百位等分别相加，并考虑低字节向高字节的进位)。选用工作寄存器暂存中间积。

问题分析：

设被乘数低字节（addrl）用A表示，高字节（addr2）用B表示；乘数低字节（addr3）用L表示，高字节（addr4）用M表示。结果存放在addr1~addr4(第字节在前)

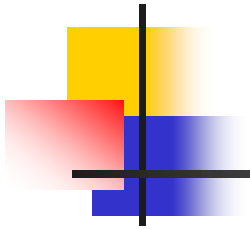
双字节相乘的过程如下：

			B	A	；被乘数
		×	M	L	；乘数
			HAL	LAL	；(1)部分积
		HBL	LBL		；(2)部分积
		HAM	LAM		；(3)部分积
＋	HBM	LBM			；(4)部分积
RES3(R4)		RES2(R3)	RES1(R2)	RES0	；积

工作寄存器用来存放部分积，R2存放(HAL + LBL + LAM)，R3存放(HBL +CY + HAM + LBM)，R4存放(HBM十CY)。双字节乘法程序段如下：

```
START：PUSH     PSW          ；PSW、A、B入栈
       PUSH     ACC
       PUSH     B
       MOV      PSW，＃18H ；选用工作寄存器组 3
       MOV      R0，＃addrl  ；被乘数低字节地址送R0
       MOV      R1，＃addr3  ；乘数低字节地址送R1
       MOV      A，@R0       ；被乘 数低字节内容送A
       MOV      B，@R1       ；乘数低字节内容送B
       MUL      AB           ；(1) A×L
       MOV      @R0，A       ；积的最低字节存入addrl中
```

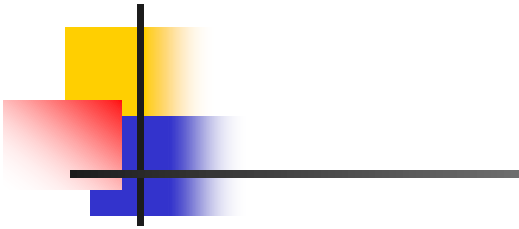
```
MOV     R2，B       ；HAL送R2中
INC     R0          ；指向被乘数高字节
MOV     A，@R0      ；被乘数高字节送A
MOV     B，@R1      ；乘数低字节送B
MUL   AB            ；(2) B×L
ADD     A，R2       ；HAL + LBL
MOV     R2，A       ；HAL + LBI之和送R2
MOV     A，B        ；HBL送A
ADDC    A，＃00H    ；HBL + CY
MOV     R3，A       ；HBL送R3
DEC     R0          ；指向ddrl
MOV     A，@R 0     ；A送累加器A
```



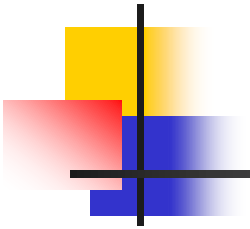
```
MOV   A，@R1          ；M送B
MUL   AB              ；(3) M×A
ADD   A，R2           ；LAM＋（R2）
MOV   R2，A           ；LAM＋HAL十LBL之和送R2
MOV   A，B            ；HAM送A
ADDC  A，R3           ；HAM十HBL＋CY
MOV   R3，A           ；HAM＋HBL＋CY之和送R3
MOV   R4，＃0         ；清R4
JNC           LOOP    ；判CY，CY＝0转LOOP
INC   R4              ；CY＝1 则（R4）←R4＋I
```



```
LOOP：INC   R0          ；指向addr2
       MOV  A，@R0      ；B送 A
       MOV  B，@R1      ；M送B
       MUL  AB          ；(4) M×B
       ADD  A，R3       ；HAM十HBL＋LBM
       MOV  R3，A       ；HAM＋HBL＋LBM之和送R3
       MOV  A，B        ；HBM送A
       ADDC A，R4       ；R4＋HBM＝RES3
       MOV  @R1，A      ；RES3存入 addr4中
       MOV  @R0、R2     ；RES1存入addr2中
```



```
DEC    R1          ；指向addr3
MOV   @R1,R3       ；RES2存入addr3中
POP    B           ；B、A、PSW出栈
POP    ACC
POP    PSW
```



例4.2.1-5

例: 双字节无符号数除法

1). 除法指令DIV AB是条单字节除法

2). 多字节无符号数的除法，可以依照“ 移位相减” 的基本方法来进行。

二进制的除法类似十进制的除法:

除法运算是按位进行的，每一位是一个循环，每个循环中做三件事，一是被除数左移一位，二是余数减除数，最后根据是否够减来置商位为“1”或“0”。

对于16位的被除数，要循环16次才能完成除法运算。若除数为零，则除法无法进行，这时置溢出标志为“1”。



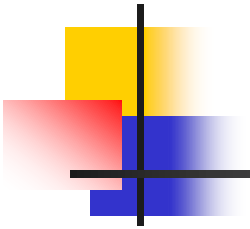
举例：8位二进制的除法, 10011101 / 0000 1100

```
      095   商数
56 | 5324
     504
     ---
     028 4
      280
      ---
        4   余数
```

十进制除法：移位相减

```
        0 1101   商数
1100 | 1001 1101
       110 0
       -----
       011 11
        11 00
        -----
        00 1101
           1100
           ----
           0001  余数
```

二进制除法：移位相减



对于被除数的移位，最简单的办法是把被除数向余数单元左移，把被除数左移后空出的低位存放商数，当除法完成后，被除数已全部移到余数单元并逐次被减而得到余数，而被除数单元中内容已成为商数。

程序部分略。

例4.2.1-6

例： 查表

根据变量x，在表格中查找对应的y值，使y = f（x）。y与x的对应关系可有各种形式，而表格也可有各种结构。

设有一个巡回检测报警装置，需对4 路输入进行控制，每路设有一个最大额定值，为双字节数。控制时需根据检测的路号找出该路对应的最大额定值。设R2用于寄存检测路号，查找到的对应的最大额定值存放于31H和32H单元中。查找最大允许额定值子程序如下：

	源程序		注释
	ORG 1000H		
1000 EA	MOV A,R2	;	检测路号送A
1001 2A	ADD A,R2	;	（R2）×2
1002 F531	MOV 31H,A	;	距表首址偏移量
1004 2408	ADD A, **#08H**	;	**偏移量 = 000FH-0007H**
1006 83	**MOVC A,@A+PC**	;	查表，读取第一个字节内容
1007 C531	XCH A,31H	;	第一字节存入31H单元
1009 2404	ADD A, **#04H**	;	**偏移量 = 000FH+1-000CH**
100B 83	**MOVC A,@A+PC**	;	查表，读取第二字节
100C F532	MOV 32H,A	;	第二字节存入32H单元
100E 22	RET		
100F 1230	DW 1230H,1540H	;	最大额定值表：路号0，路号1
1011 1540			
1013 2340	DW 2340H,2430H	;	路号2，路号3
1015 2430			
	END		

以DPTR为基址寄存器

```
        ORG 0000H
        MOV A, R2       ;
        ADD A, R2       ; (R2)×2
        MOV R2, A       ;存放被查找数与表首地址的偏移量
        MOV DPTR, #TAB1;取表首地址
        MOVC A, @A+DPTR;读第一字节
        MOV 31H, A      ;存放第一字节
        INC R2          ;偏移量+1
        MOV A, R2       ;
        MOVC A, @A+DPTR;读第二字节
        MOV 32H, A      ;存放第一字节
        RET
TAB1:   DW 1230H, 1540H;最大额定值表：路号0，路号1
        DW 2340H, 2430H;路号2，路号3
        END
```

例4.2.1-7

例： 查表

设表中有1024个元素，每个元素为两个字节，则表格总长为2048个字节。现按R4和R5的内容从表格中查出对应的数据元素值，送存R4和R5中。其程序如下：

问题分析：

根据R4R5组成16位数查找表格中对应的元素(<03FFH)

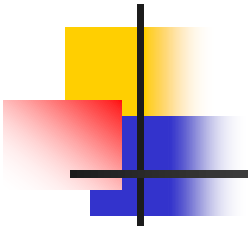
求解问题：

R4R5给定的偏移量需要乘2（每个元素是2个字节）

可以通过左移1位实现，R5 左移考虑进位影响

表首地址以DPTR给定

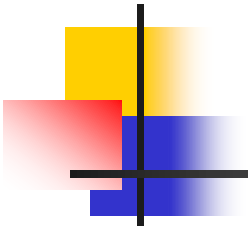
```
TBDP1: MOV   DPTR，＃addrl6；表格首地址值送DPTR
       MOV    A，R5          ；查表参数低位字节送A
       CLR    C              ；清CY
       RLC    A              ；带进位左移一位, A=(R5)*2
       XCH    A，R4          ；将查表参数R4内容送A=R4, R4=R5*2
       RLC    A              ；带进位左移一位, A=R4*2，考虑了进位
       XCH    A，R4          ；R4与R5内容互换, R4=R4*2, A=R5*2
       ADD    A，DPL         ；表首地址低字节＋偏移量低字节
       MOV    DPL，A         ；更新DPL
       MOV    A，DPH         ；表首地址高字节＋偏移量高字节，考虑进位
       ADDC   A，R4          ；
       MOV    DPH，A         ；相加和存DPH，更新DPH
       CLR    A              ；清A
       MOVC   A，@A＋DPTR    ；查表，读第一字节
       MOV    R4，A          ；第一字节存入R4
       CLR    A              ；清A
       INC    DPTR           ；（DPTR）＋1
       MOVC   A，@A＋DPTR    ；查表，读第二字节
       MOV    R5，A          ；第二字节存入R5
       RET
addr16: DW    …              ；数据表
       DW    …
```



4.2.2 分支结构程序设计：

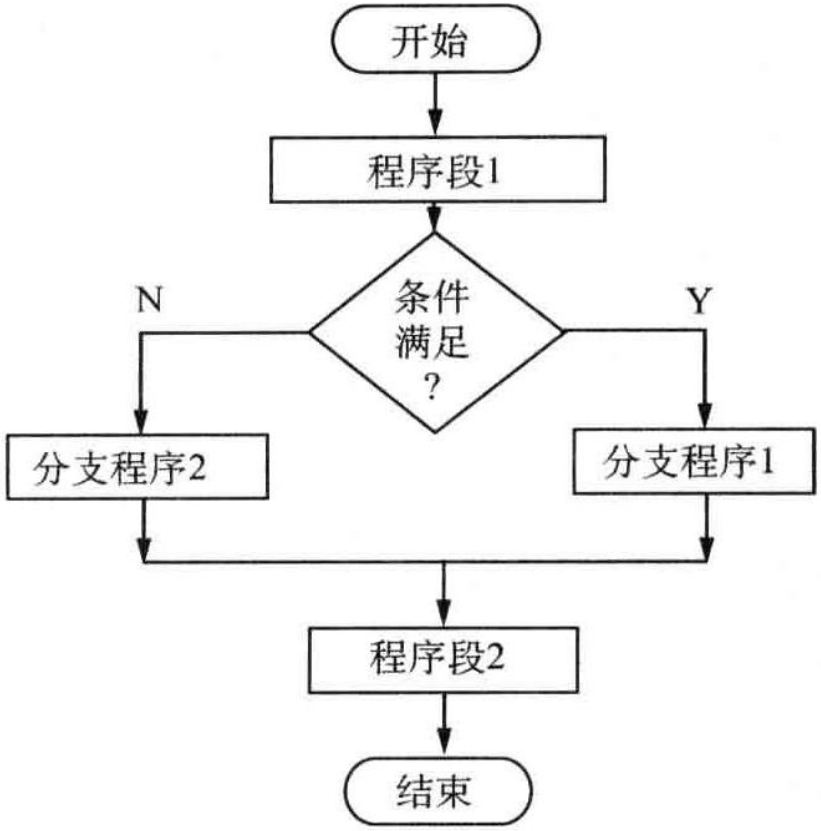
特点:

- **程序执行流程中必然包含有条件判断。**
- **符合和不符合条件有不同的处理路径**
- **单分支结构和多分支结构**
- **单分支结构采用条件转移指令，多分支需要多个判断转移指令组合使用**

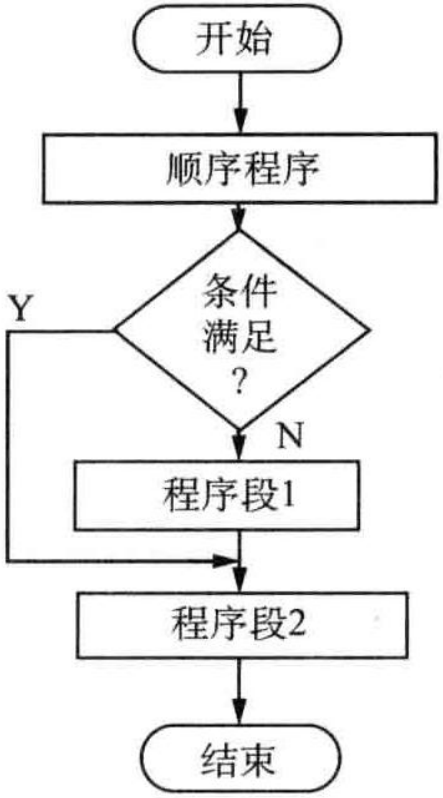


➢ **单分支结构：**

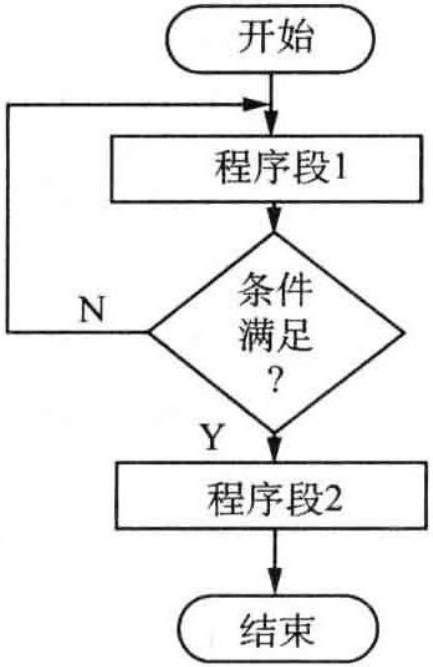
- 程序仅有两个出口，两者选一。通常用条件判断指令来选择并转移。
- 如：JC、JNC、JB、JNB和JBC等，还有一些条件转移指令，如：JZ、JNZ、DJNZ等。
- 这类单分支结构程序有三种典型的形式：



(a)形式1



(b)形式2



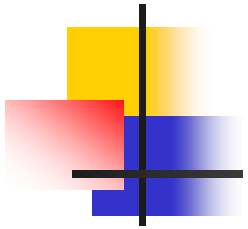
(c)形式3

例：求双字节补码程序。

设对addrl，addrl+1的双字节读数（已知是负数）取补后存入addr2和 addr2+1单元中，其中高位字节在高地址单元中。8位微机对双字节数取补需分二次进行。首先对低字节数取补，然后判其结果是否为全“0”。若为“0”，则高字节数取补；否则，高位字节数取反。

说明：

- 低8位取补后低字节全”0”，表示有进位，所以高位需要取反加1
- 低8位取补后低字节非全”0”，高位取反无需加1 （低位已经加1）



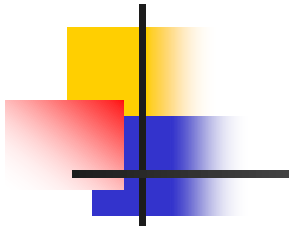
```
START: MOV   R0，＃addrl     ；原码低字节地址码送R0
       MOV   R1，＃addr2     ；补码低字节地址码送R1
       MOV   A，@R0          ；原码低字节内容送A
       CPL   A
       INC   A               ；A内容取反加l，即取补
       MOV   @R1，A          ；低字节补码存addr2单元
       INC   R0              ；指向原码高字节
       INC   R1              ；指向补码高字节
       JZ    LOOP1           ；当（A）＝0，转LOOP1
       MOV   A，@R0          ；原码高字节送A
       CPL   A               ；高字节内容取反
       MOV   @R1，A          ；字节反码存(addr2十1)单元
       SJMP  LOOP2           ；转LOOP2，结束
LOOP1: MOV A，@R0            ；低字节补码为0
       CPL   A               ；对高字节数取补
       INC   A
       MOV   @R1，A          ；高字节补码存(addr2+1)单元
LOOP2: …
       END                   ；结束
```

例: 试编写计算下式的程序

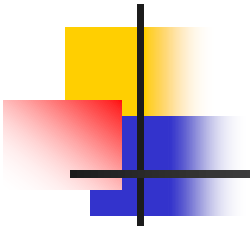
$Y=a^2+b$ (当b≥10时)

$Y=a^2-b$ (当b<10时)

```
        ORG    0000H
START:  MOV    A, #a
        MOV    B, A
        MUL    AB              ; BA= a*a
        MOV    R0,A            ; R1R0= a*a
        MOV    R1,B
        MOV    A, #b
        CJNE   A,#0AH,MMN      ; b≠10则转移
MM:     ADD    A,R0            ; b≥10, a2+b=Y
        MOV    R0,A            ; 保存到R0
        MOV    A,#00H          ; 需要考虑低位向高位的进位
        ADDC   A,R1
        MOV    R1,A
        AJMP   MMNN
```



```
MMN：JNC     MM              ；无借位（即b>10）转MM
       MOV   R3,A            ；R3←b
       MOV   A，R0
       CLR   C
       SUBB  A,R3            ；  R1R0←a2 - b
       MOV   R0,A
       MOV   A,R1            ；需要考虑低位向高位的借位
       SUBB  A,#00H
       MOV   R1,A
MMNN：MOV Y0，R0             ；Y1Y0←结果
       MOV   Y1,R1
HERE：AJMP  HERE
```



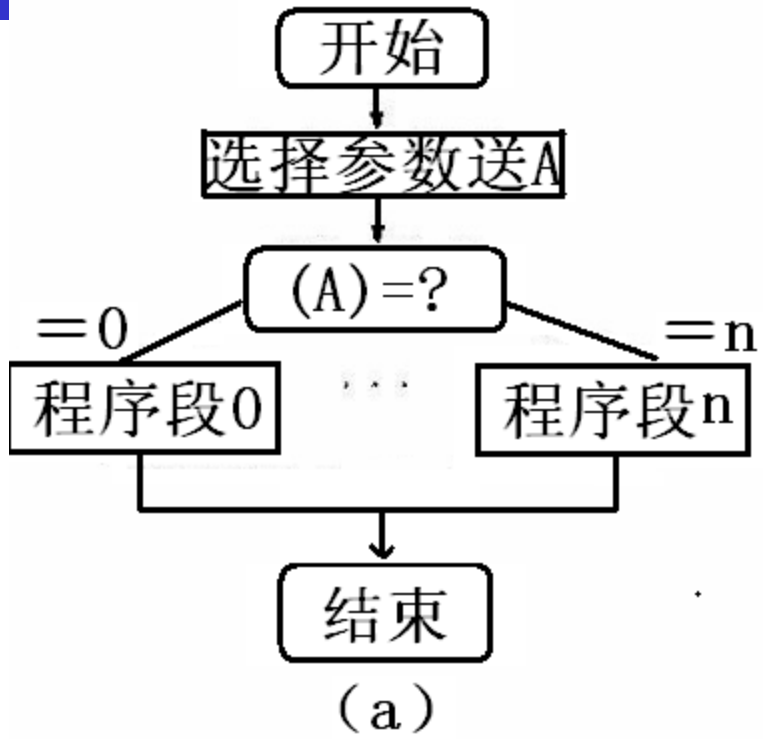
➢ 多分支选择结构

- 当程序的判别部分有两个以上的出口流向时，称为多分支结构。
- 80C51的多分支选择指令：

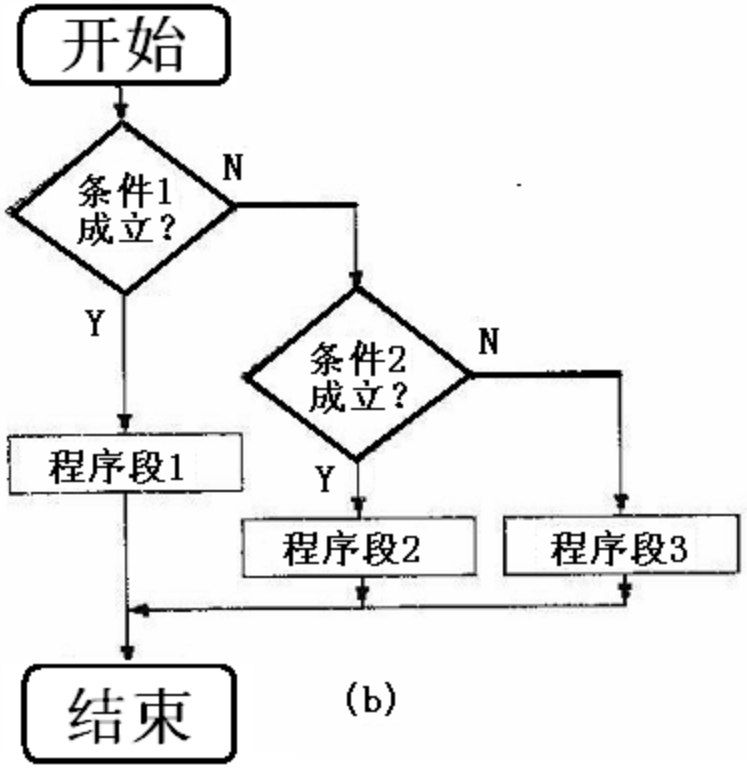
  JMP @A+DPTR

  CJNE A

- 分支结构程序允许嵌套
- 多种实现方式



(a)



(b)

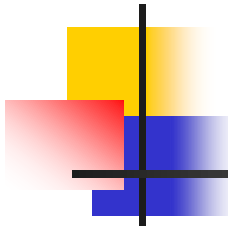
多分支结构通常有两种形式，



1） 通过堆栈操作实现多分支程序转移

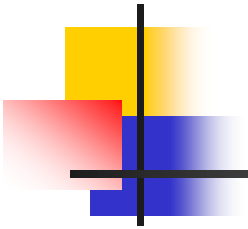
通过堆栈操作，把分支程序的入口地址压入堆栈中，然后利用返回指令把分支程序的入口地址装入PC,实现了分支程序的转移。

例： 假定分支程序的入口地址表首址为BRTAB，分支转移序号在R3中



```
        MOV    DPTR,#BRTAB ；分支入口地址表首地址
        MOV    A,R3
        RL     A           ；×2
        MOV    R1,A        ；暂存A（分支值乘2）
        INC    A           ；取低位地址
        MOVC   A,@A+DPTR   ；
        PUSH   A           ；低位地址入栈
        MOV    A,R1
        MOVC   A,@A+DPTR   ；取高位地址，并入栈
        PUSH   A
        RET                ；分支入口地址出栈送入PC
BRTAB: DW  BR0, BR1, BR2, BR3
BR0: -----                 ；略
BR1: -----
BR2: -----
```

注意DW存放高低字节的顺序以及堆栈操作需要的高低字节的顺序



2) 通过查转移指令表的方法实现多分支程序转移

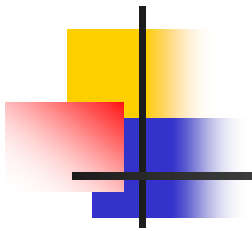
用转移指令构成一个表

由40H单元中动态运行结果值来选择分支程序

（40H）＝0，转处理程序0

（40H）＝1，转处理程序1

（40H）＝n，转处理程序n

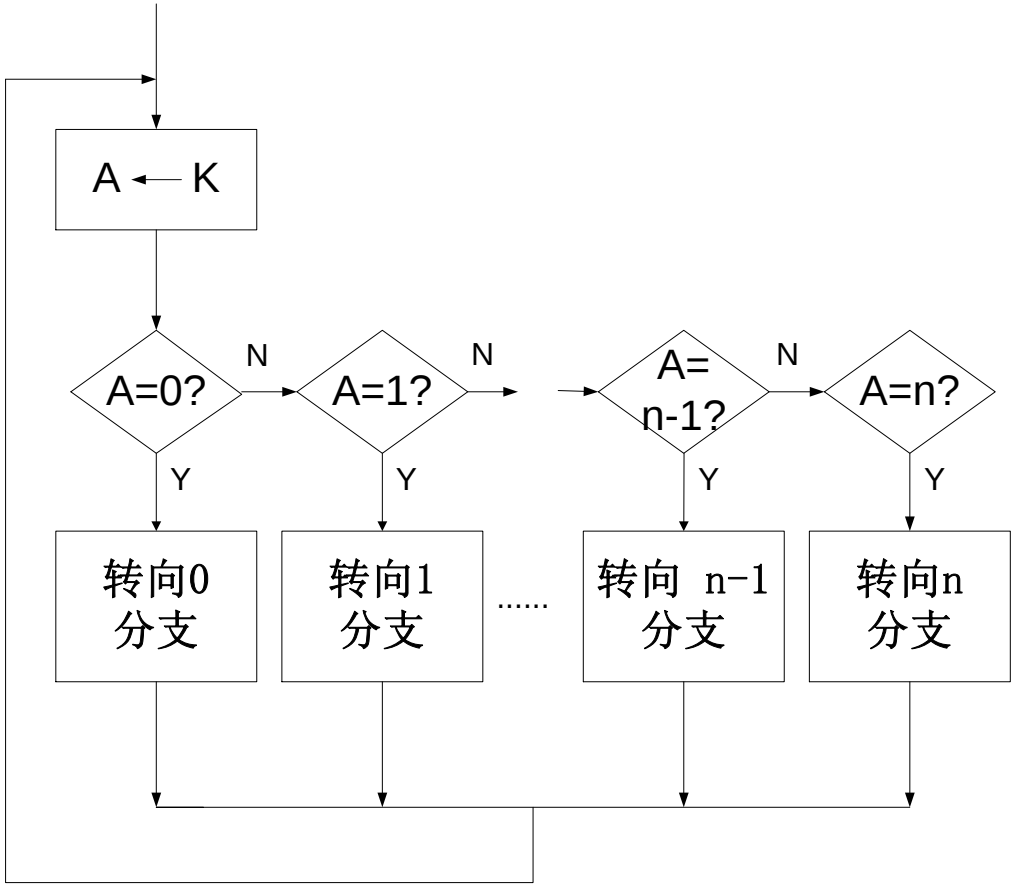


```
START：MOV  DPTR，＃ADDR16；多分支转移指令表首址送DPTR
        MOV    A，40H          ；40H单元内容送A
        CLR    C               ；清CY序
        RLC    A               ；A内容左移一位，×2
        JNC    TABLE           ；若CY＝0转TABEL
        INC    DPH             ；若CY＝l，DPH内容十1
TABLE: JMP  @A+DPTR            ；多分支转移
ADDR16: AJMP  LOOP0            ；转分支程序0
        AJMP  LOOP1            ；转分支程序1
                …
        AJMP  LOOPn            ；转分支程序n
```

AJMP的每条指令占用两个字节，分支的值需要×2，进位加到DPH

转移的范围：2K （如果采用LJMP，可以达到64K)

3) 使用CJNE指令逐次比较



例13. 对从P1口输入的100个0～9的数进行概率统计。统计的数值分别存入20H～29H中。

```
        CLR   A              ；结果单元20H～29H清零
        MOV   R0,#10
        MOV   R1,#20H
LP:     MOV   @R1,A
        INC   R1
        DJNZ  R0,LP
        MOV   R0,#100        ；100个数的计数器
READ:   MOV   A, P1          ；读入P1
CHK0:   CJNE  A,#0,CHK1      ；比较，不为“0”则继续比较
        INC   20H            ；是“0”，则“0”计数单元加1
        SJMP  END0           ；调转到判断是否全部统计完
CHK1:   CJNE  A,#1,CHK2
        INC   21H            ；是“1”，则“1”计数单元加1
        SJMP  END0
```

CHK2 --- CHK8:



```
CHK9：CJNE   A,#9,ERR        ;比较，不为“9”则报错
      INC    29H             ；是“9”，则“9”计数单元加1
END0：DJNZ   R0,READ         ；判是否全部统计完?
HERE：SJMP   HERE
ERR：…                       ；非0～9，出错
```

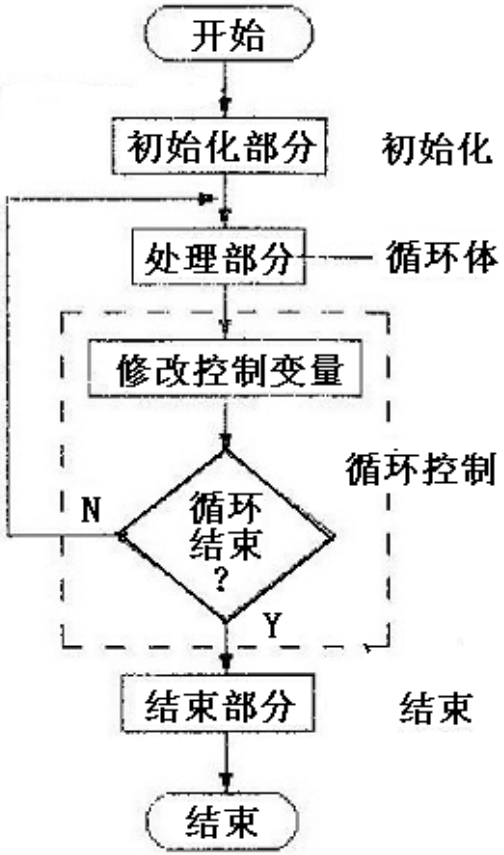
4.2.3 循环结构程序设计：

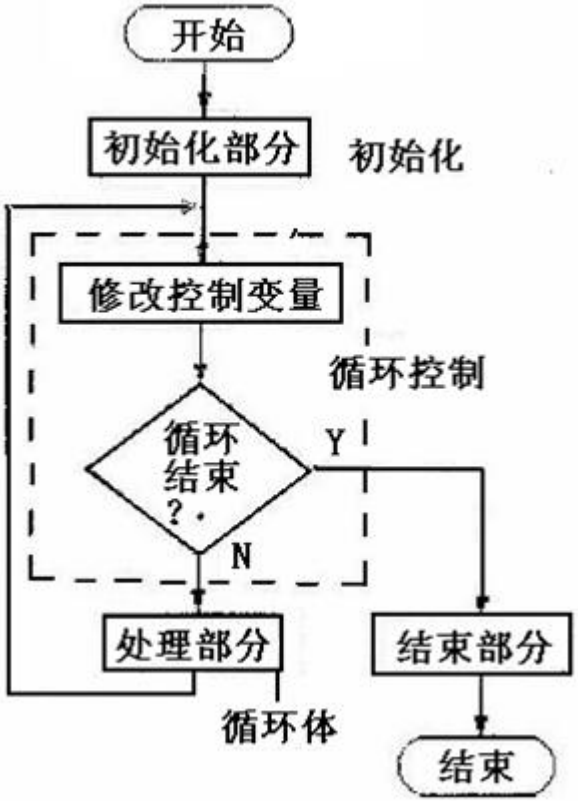
特点：

- **重复多次地执行一串指令的基本程序结构。**
- **只是分支程序中的一个特殊形式**
- **四个组成部分：**

- **循环初始部分**：进入循环程序体之前所必要的准备工作
- **循环体：**循环结构程序的核心部分，完成实际的处理工作，是需反复循环执行的部分
- **循环控制部分：**

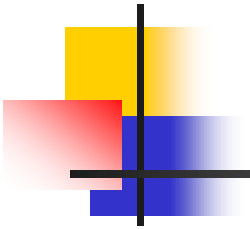
  控制循环程序的循环与结束部分，如计数、条件判断
- **循环结束部分**：对循环程序执行的结果进行分析、处理和存放





计数循环结构：至少执行1次

条件循环结构：可以1次都不执行



➢ 计数控制循环结构

特点:

- 循环次数已知，必须在初始化部分设定计数的初值，
- 循环控制部分依据计数器的值决定循环次数。
- 常用指令：DJNZ Rn, rel ; DJNZ direct, rel

例: 软件延时, 延时子程序:

```
DELAY:    MOV  R2, #data      ; 预置计数循环控制常数
DELAY1:   DJNZ  R2，DELAY1 ;  当（R2）≠0，转向本身
          RET
```

根据R2的不同初值可实现3～513个机器周期的延时（第一条为单周期指令，第二条为双周期指令，返回指令不计）。

** 掌握计算软件延时程序时间的方法

例：单字节16进制（<99）转换为BCD码

;;输入：R0 (<99)　　;;输出：R1

HexToBCD:

编程原理：设计循环体

Byte ----bit7 bit6 bit5 bit4 bit3 bit2 bit1 bit0

第0次：　bit7

第一次：　bit7*2+bit6

第二次：　(bit7*2+bit6) *2+bit5

第三次：((bit7*2+bit6) *2+bit5)*2+bit4

第四次：(((bit7*2+bit6) *2+bit5)*2+bit4)*2 +bit3

第五次：((((bit7*2+bit6) *2+bit5)*2+bit4)*2 +bit3)*2+bit2

第六次：(((((bit7*2+bit6) *2+bit5)*2+bit4)*2 +bit3)*2+bit2)*2 +bit1

第七次：((((((bit7*2+bit6) *2+bit5)*2+bit4)*2 +bit3)*2+bit2)*2 +bit1)*2 +bit0

例：单字节16进制（<99）转换为BCD码

;;输入：R0 (<99)　　;;输出：R1

```
HexToBCD:
        MOV     R2,#8           ;;循环次数
        MOV     R1,#0           ;;初始化输出
LOOP:
        MOV     A,R0
        RLC     A               ;;R0最高位移到C
        MOV     R0,A
        MOV     A,R1            ;;R1+R1+C
        ADDC    A,ACC
        DA      A               ;;变为10进制
        MOV     R1,A
        DJNZ    R2,LOOP
        RET
```

设R0=61H（97）或 0110， 0001

```
HexToBCD:
        MOV     R2,#8           ;;循环次数
        MOV     R1,#0           ;;初始化输出
LOOP:           Loop1
        MOV     A,R0
        RLC     A               ;;R0最高位移到C
        MOV     R0,A
        MOV     A,R1            ;;R1+R1+C
        ADDC    A,ACC
        DA      A               ;;变为10进制
        MOV     R1,A            ;;放回R1
        DJNZ    R2,LOOP
        RET
```

R0= 110， 0001， x

C=0

A=0

R1=0

设R0=61H（97）或 0110， 0001

```
HexToBCD:
        MOV     R2,#8          ;;循环次数
        MOV     R1,#0          ;;初始化输出
LOOP:           Loop2
        MOV     A,R0
        RLC     A              ;;R0最高位移到C
        MOV     R0,A
        MOV     A,R1           ;;R1+R1+C
        ADDC    A,ACC
        DA      A              ;;变为10进制
        MOV     R1,A           ;;放回R1
        DJNZ    R2,LOOP
        RET
```

R0= 10， 0001，xx

C=1

A=1

R1=1

设R0=61H（97）或 0110， 0001

```
HexToBCD:
        MOV     R2,#8           ;;循环次数
        MOV     R1,#0           ;;初始化输出
LOOP:   Loop3
        MOV     A,R0
        RLC     A               ;;R0最高位移到C
        MOV     R0,A
        MOV     A,R1            ;;R1+R1+C
        ADDC    A,ACC
        DA      A               ;;变为10进制
        MOV     R1,A            ;;放回R1
        DJNZ    R2,LOOP
        RET
```

R0= 0， 0001，xxx

C=1

A=3

R1=3

设R0=61H（97）或 0110， 0001

```
HexToBCD:
        MOV     R2,#8          ;;循环次数
        MOV     R1,#0          ;;初始化输出
LOOP:           Loop4
        MOV     A,R0
        RLC     A              ;;R0最高位移到C
        MOV     R0,A
        MOV     A,R1           ;;R1+R1+C
        ADDC    A,ACC
        DA      A              ;;变为10进制
        MOV     R1,A           ;;放回R1
        DJNZ    R2,LOOP
        RET
```

R0= 0001，xxxx

C=0

A=6

R1=6

设R0=61H（97）或 0110， 0001

```
HexToBCD:
        MOV     R2,#8          ;;循环次数
        MOV     R1,#0          ;;初始化输出
LOOP:   Loop5
        MOV     A,R0
        RLC     A              ;;R0最高位移到C
        MOV     R0,A
        MOV     A,R1           ;;R1+R1+C
        ADDC    A,ACC
        DA      A              ;;变为10进制
        MOV     R1,A           ;;放回R1
        DJNZ    R2,LOOP
        RET
```

R0= 001，xxxxx

C=0

A=0CH

R1=12H

设R0=61H（97）或 0110， 0001

```
HexToBCD:
        MOV     R2,#8          ;;循环次数
        MOV     R1,#0          ;;初始化输出
LOOP:           Loop6
        MOV     A,R0
        RLC     A              ;;R0最高位移到C
        MOV     R0,A
        MOV     A,R1           ;;R1+R1+C
        ADDC    A,ACC
        DA      A              ;;变为10进制
        MOV     R1,A           ;;放回R1
        DJNZ    R2,LOOP
        RET
```

R0= 01，xxxxxx

C=0

A=24H

R1=24H

设R0=61H（97）或 0110， 0001

```
HexToBCD:
        MOV     R2,#8          ;;循环次数
        MOV     R1,#0          ;;初始化输出
LOOP:   Loop7
        MOV     A,R0
        RLC     A              ;;R0最高位移到C
        MOV     R0,A
        MOV     A,R1           ;;R1+R1+C
        ADDC    A,ACC
        DA      A              ;;变为10进制
        MOV     R1,A           ;;放回R1
        DJNZ    R2,LOOP
        RET
```

R0= 1，xxxxxx

C=0

A=48H

R1=48H

设R0=61H（97）或 0110， 0001

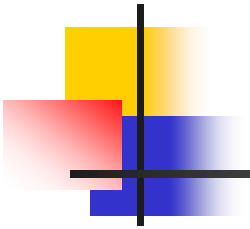
```
HexToBCD:
        MOV     R2,#8          ;;循环次数
        MOV     R1,#0          ;;初始化输出
LOOP:           Loop8
        MOV     A,R0
        RLC     A              ;;R0最高位移到C
        MOV     R0,A
        MOV     A,R1           ;;R1+R1+C
        ADDC    A,ACC
        DA      A              ;;变为10进制
        MOV     R1,A           ;;放回R1
        DJNZ    R2,LOOP
        RET
```

R0= xxxxxxx

C=1

A=91H

R1=97H



➢ 条件控制循环结构

特点:

- 根据控制循环结束的条件，决定是否继续执行循环程序
- 常用指令：比较类指令、条件判断类指令

例4.2.3-2：把内部RAM中起始地址为DATA的数据串传送到外部RAM以BUFFER为首地址的区域，直到发现“$”字符的ASCII码为止，数据串的最大长度在内存20H中。

```
        MOV    R0,#DATA          ；数据区首地址
        MOV    DPTR,#BUFFER      ；数据区长度指针
LOOP:   MOV    A,@R0
        CJNE   A,#24H，LOOP2     ；判是否为“$”符(24H)
        SJMP   LOOP1             ；是“$”符，则结束
LOOP2:  MOV    A,@R0             ；不是“$”符，则传送
        MOVX   @DPTR,A
        INC    R0
        INC    DPTR
        DJNZ   20H,LOOP          ；数据串未查完，继续
LOOP1:  RET
DATA:   …                        ；数据串
```

注：本题中循环控制条件有二个，一个是条件循环控制，以找到ASCII码“$”符为循环结束条件，这是主要的结构；第二个是计数循环结构，万一找不到ASCII码“$”符，则由数据串的最大长度作为计数循环控制。



4 循环嵌套结构

循环嵌套就是在循环内套循环的结构形式，也称多重循环。

循环的执行过程是从内向外逐层展开的。内层执行完全部循环后，外层则完成一次循环，逐次类推。层次必须分明，层次之间不能有交叉，否则将产生错误。

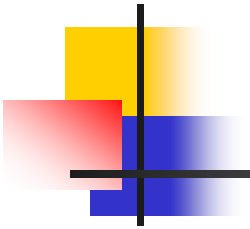
```
DELAY2:MOV     R5，#TIME1 ；外循环次数送R5 机器周期1
LOOP2:  MOV     R4，#TIME2 ；内循环次数送R4 机器周期1
LOOP1:  NOP                ；空操作 机器周期1
        NOP                ； 机器周期1
        DJNZ   R4，LOOP1   ；内循环次数-1=0?
                           ；不为0循环，机器周期2
        DJNZ   R5，LOOP2   ；外循环次数-1=0?
                           ；不为0则循环 机器周期2
        RET                ； 机器周期2
       END
```

4.2.4 子程序设计：

特点：

- **需要被反复调用的程序段适合作为专门的子程序**
- **标准化，具有一定的通用性**
- **子程序一定需要一个入口地址（第一条命令），通常用标号表示**
- **子程序一定以RET结尾**

调用子程序的程序称为主程序或调用程序



- **子程序调用指令**

1) 绝对调用指令：ACALL addr11

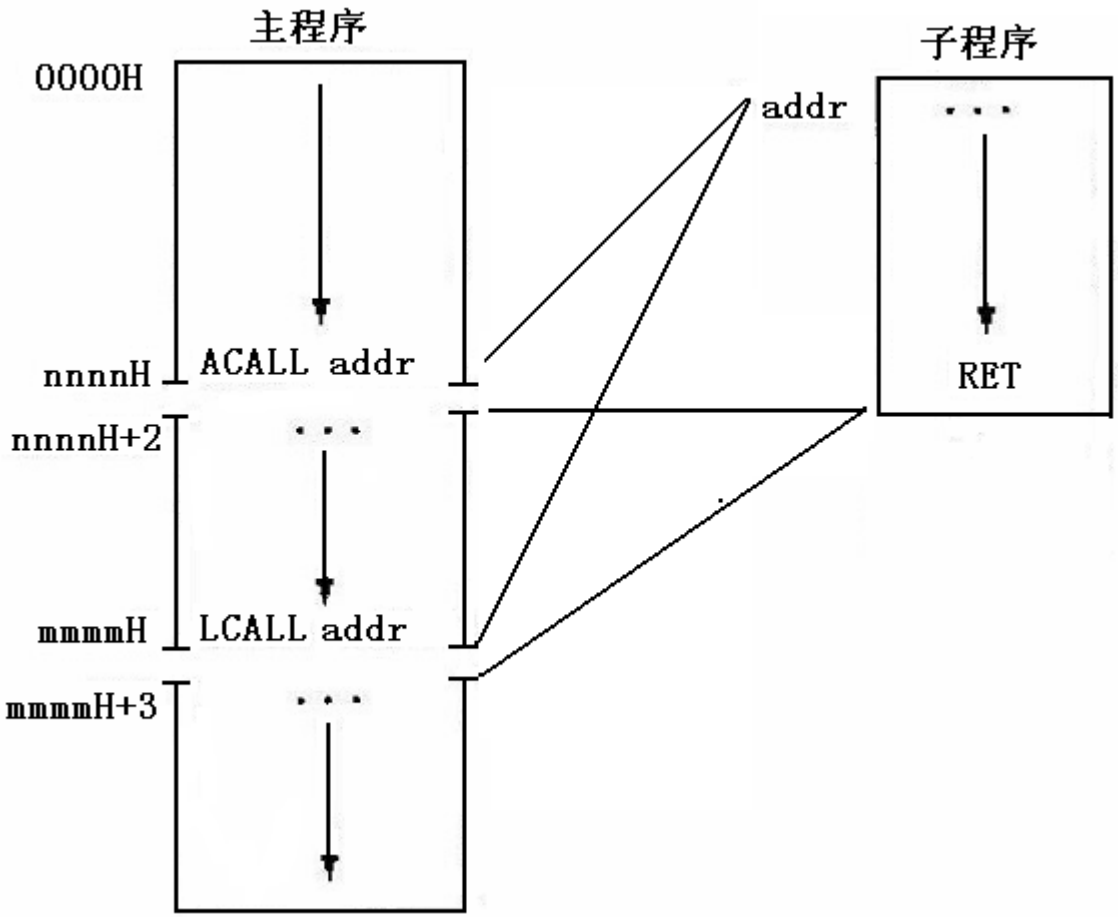
二字节指令，PC低11位被改变，寻址空间2K

2) LCALL addr16

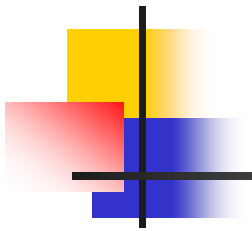
三字节指令，PC被重新赋值，寻址空间64K

3) RET

在子程序末尾，返回断点（调用指令的下一条指令处）


主程序
子程序
0000H
addr
nnnnH
ACALL addr
nnnnH+2
RET
mmmmH
LCALL addr
mmmmH+3

子程序调用示意图



- **子程序嵌套**

子程序中调用其他子程序，类似循环嵌套

需要注意堆栈的使用，返回点的考虑

使用多重嵌套时，注意防止逻辑混乱及堆栈溢出



- **子程序设计注意点：**

1) 子程序入口条件

需要数据（参数）传输的场合，需要有子程序与主程序共享的存储单元

2）保护现场与恢复现场

单片微机只是自动保护断点地址，其他数据的保护需要用指令完成

3）详细描述子程序的特性

参数，使用的寄存器，实现的功能等

错误？

```
;;延时100ms
DELAY:
        MOV     R0，#100          ；设置循环次数
LOOP2:
        LCALL   DELAY1MS          ；调用子程序延时1ms
        DJNZ    R0，LOOP2         ；  循环次数-1=0?
        RET
;;延时1ms
DELAY1MS:
        MOV   R0,#250
LOOP1:  NOP                       ；空操作 机器周期1
        NOP
        DJNZ    R0，LOOP1         ；内循环次数-1=0?
        RET
```

例4.2.4－1

例： 四位十进制数转换成二进制整数子程序。

十进制数放在低4位，高4位为0

入口参数：BCD码字节地址指针R0、位数(n－1)存于R2中。

出口参数：二进制数存放于R3、R4中。

目标：$A = a3\times10^3+a2\times10^2+a1\times10^1+a0\times10^0$

实现：调用子程序，子程序用循环实现N位转换

$$A=\{[(a3\times10+a2)]\times10+a1\}\times10+a0\}$$

算法：当前位乘以10 加上下一位

主程序如下:

```
ORG 0000H
MOV SP, #30H;
MOV R0, #60H;
MOV R2, #03H; 循环次数
MOV 60H, #05H; BCD码为5088
MOV 61H, #00H;
MOV 62H, #08H;
MOV 63H, #08H;
LCALL BCDB; 调用转换子程序
NOP
SJMP $
```



子程序实现算法：

•结果存放在R3, R4中，循环一次实现乘以10加下一位数字的算法的结果保存在R3, R4中

•实现需要多字节的乘法，被乘数位2字节，乘数位1字节

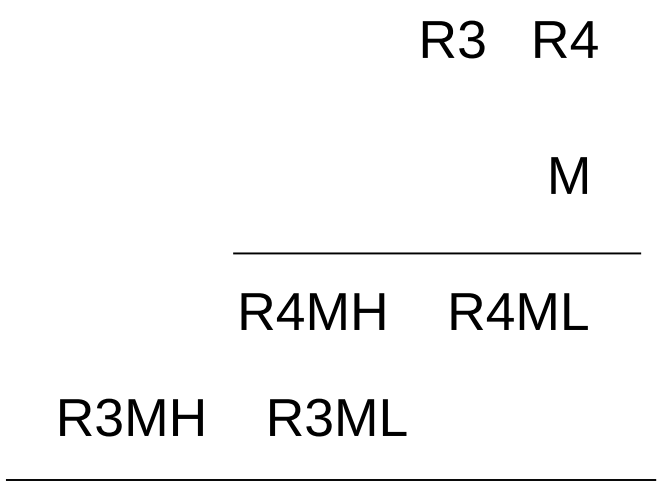
•第一次循环时高位R3为0

•为什么被乘数是2字节而不是其他?

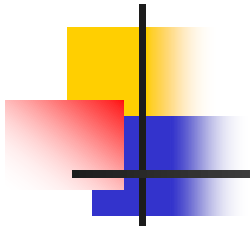
因为题目要求是4位BCD数字，最多9999，16位完全可以表示

如果要求改为5位BCD码，被乘数应该扩展为24位，结果应该存放3个字节，99999 = 1869FH，需要3个字节的乘法

```
BCDB: MOV R3, #00H;
      MOV A, @R0;
      MOV R4, A;
LOOP: MOV A, R4;
      MOV B, #10; 实现R4*M
      MUL AB
      MOV R4, A; R4=R4ML
      MOV R5, B; R5=R4MH
      MOV B, #10;
      MOV A, R3; A=R3
      MUL AB;    实现R3*M
      ADD A, R5; A=R4MH+R3ML
      MOV R3, A; R3=R4MH+R3ML，高位
```



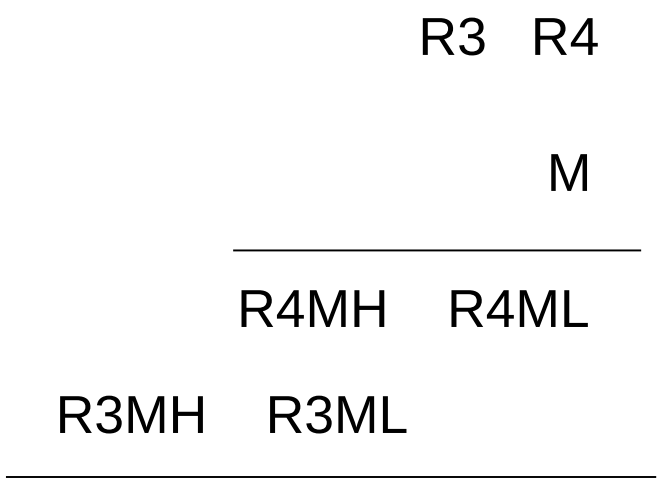
R3MH为0不予以考虑



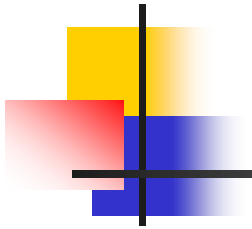
```
MOV A, R4;    低位送累加器，为多字节加法做准备
INC R0;       指针指向下一个数字
ADD A, @R0;  相加
MOV R4, A;    保存结果到R4
MOV A, R3;   高位相加，考虑进位
ADDC A, #0;
MOV R3, A;  保存高位结果到R3
DJNZ R2, LOOP;  循环
RET
END
```

方法2：将（R3 R4）×10+ @R0）作为子程序，程序如下：

```
;;输入：R0-输入数据指针；R3，R4-上次计算结果
;;输出： R3，R4-计算结果
;;使用变量：A，B，R5
BCDA: MOV A, R4
      MOV B, #10; 实现R4*M
      MUL AB
      MOV R4, A; R4=R4ML
      MOV R5, B; R5=R4MH
      MOV B, #10;
      MOV A, R3; A=R3
      MUL AB;   实现R3*M
      ADD A, R5; A=R4MH+R3ML
      MOV R3, A; R3=R4MH+R3ML，高位
```



R3MH为0不予以考虑



```
MOV A, R4;   低位送累加器，为多字节加法做准备
ADD A, @R0;  相加
MOV R4,A ;   保存结果到R4
MOV A, R3;   高位相加，考虑进位
ADDC A, #0;
MOV R3, A;  保存高位结果到R3
RET
```

主程序如下：

```
        ORG 0000H
        MOV SP, #30H;
        MOV R0, #60H;
        MOV R2, #03H; 循环次数
        MOV 60H, #05H; BCD码为5088
        MOV 61H, #00H;
        MOV 62H, #08H;
        MOV 63H, #08H;
        MOV  R4，60H
        MOV  R3，#0
LOOPA：
        INC    R0
        LCALL BCDA; 调用转换子程序
        DJNZ  R2，LOOPA
        SJMP $
```

例4.2.4－3

例： 码制转换

把外部RAM 30H～3FH共 16个单元中的ASCII码依次转换为十六进制数，并拼装压缩存入内部60H～67H共8个单元中。

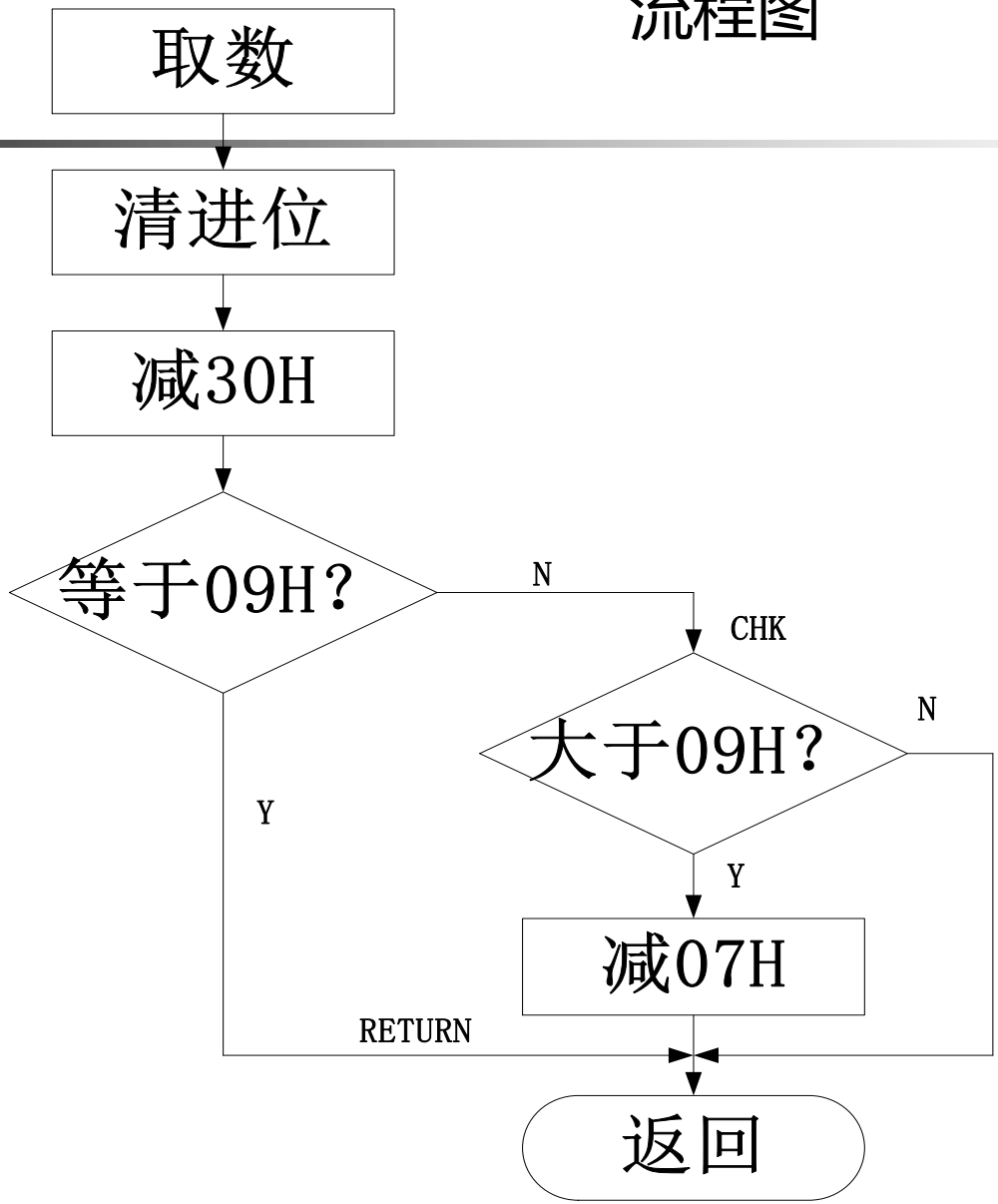
ASCII码	十六进制数	转换方法
30H~39H	0~9	ASCII码-30H（≤9）
41H~46H	A~F	ASCII码-30H-07H

两个ASCII码对应一个字节。所以在转换后再将两个高4位为0的十六进制数拼装成一个字节。（低地址单元中的ASCII码是十六进制数高位）

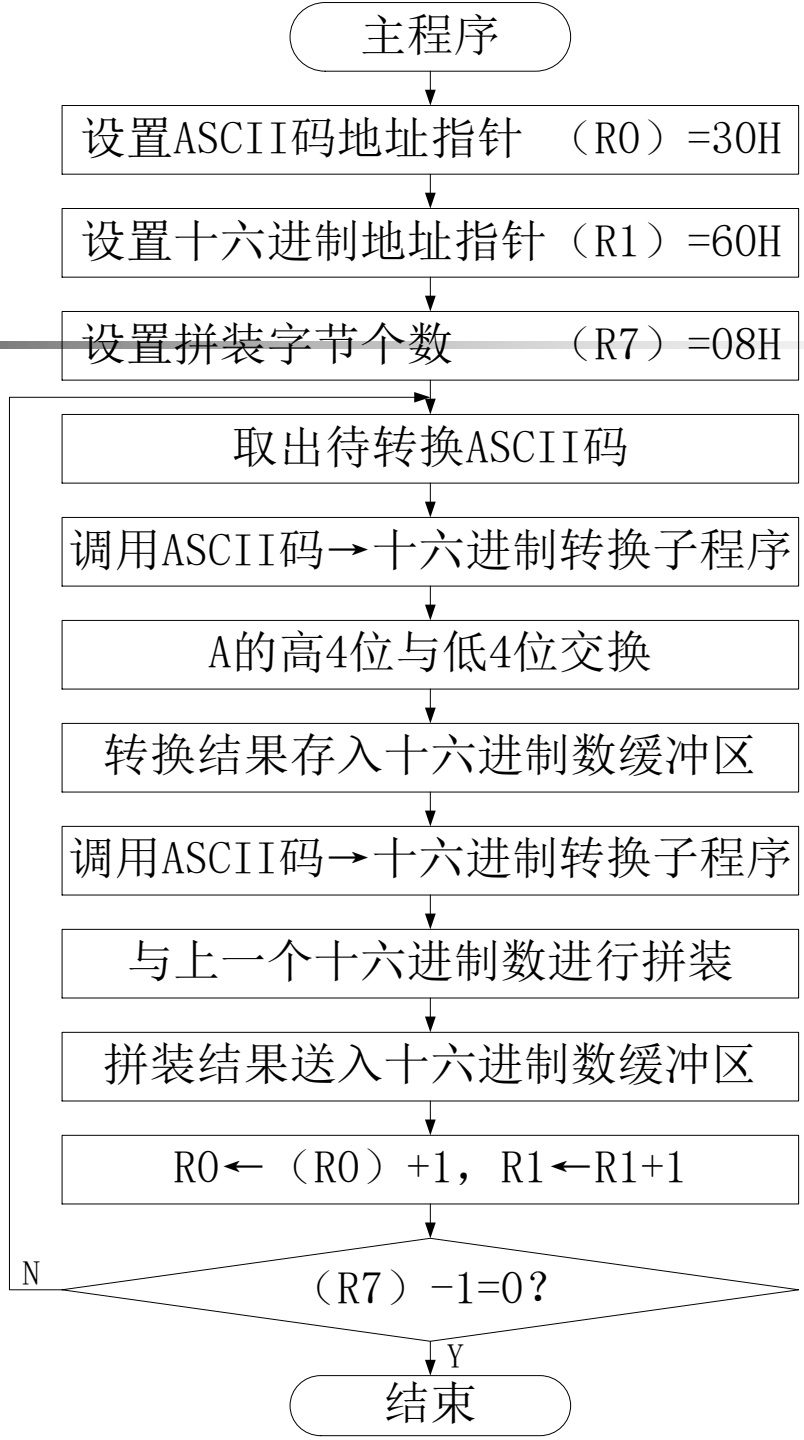
子程序：ASCII码转换为十六进制数

主程序：拼装、调整地址指针、计数

ASCII码转换为十六进制数流程图



主程序流程图


主程序
设置ASCII码地址指针 （R0）=30H
设置十六进制地址指针（R1）=60H
设置拼装字节个数 （R7）=08H
取出待转换ASCII码
调用ASCII码→十六进制转换子程序
A的高4位与低4位交换
转换结果存入十六进制数缓冲区
调用ASCII码→十六进制转换子程序
与上一个十六进制数进行拼装
拼装结果送入十六进制数缓冲区
R0←（R0）+1，R1←R1+1
（R7）-1=0？
N
Y
结束

;;输入：@R0---ASC码

;;输出：A-----数字

```
TRAN: MOVX   A，@R0        ；从外部RAM取数
      CLR    C             ；清借位
      SUBB   A，#30H       ；待转换数减30H
      CJNE   A，#09H，BB   ；结果是09H吗?
      AJMP   DONE          ；结果是09H，结束
BB:   JC     DONE          ；结果小于09H，结束
      SUBB   A，#07H       ；结果大于09H，再减07H
DONE: RET                  ；
```



```
        ORG    0000H
MAIN: MOV   SP，#70H     ； 设置堆栈区
      MOV   R0，#30H     ； 设置ASC码地址指针
      MOV   R1，#60H ； 设置十六进制数地址指针
      MOV   R7，#08H； 需拼装的十六进制数字节个数
AB1：  ACALL   TRAN          ； 调用转换子程序
      SWAP    A              ； A高低4位交换
      MOV     @R1，A   ； A的内容存放在内部RAM
```



```
        INC     R0          ；指向下一个ASCII码地址
        ACALL   TRAN        ；调用转换子程序
        XCHD    A，@R1      ；十六进制数拼装
        INC     R0          ；调整ASCII码地址指针
        INC     R1          ；调整十六进制数地址指针
        DJNZ    R7，AB1     ；判断转换是否结束？未结束则循环
HALT:   AJMP    HALT
```

4.3 程序设计举例

- 数据排序
- 数据极值查找
- 数据搜索

数据排序

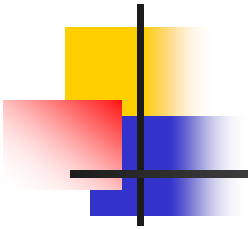
- 多种算法：

常用的有插入排序法、冒泡排序法、快速排序法、选择排序法、堆积排序法、二路归并排序法及基数排序法等

- **掌握冒泡法的算法以及程序设计**

- 冒泡法算法简介：

一种相邻数互换的排序方法，其过程类似水中气泡上浮。执行时从前向后进行相邻数比较，若数据的大小次序与要求的顺序不符时(逆序)，就将二数互换，正序时不交换



假定原始数据为 5F，38，7，13，44，D8，22

第一次冒泡过程：5F，38，7，13，44，D8，22（逆序，则二数互换）

38，5F，7，13，44，D8，22（逆序，则二数互换）

38，7，5F，13，44，D8，22（逆序，则二数互换）

38，7，13，5F，44，D8，22（逆序，则二数互换）

38，7，13，44，5F，D8，22（正序）

38，7，13，44，5F，D8，22（逆序，则二数互换

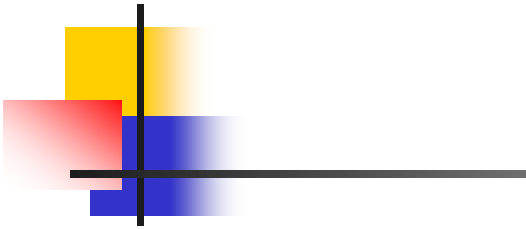
38，7，13，44，5F，22，D8(第一次冒泡结束)

第二次冒泡结束：7，13，38，44，22，5F，D8

第三次冒泡结束：7，13，38，22，44，5F，D8

第四次冒泡结束：7，13，22，38，44，5F，D8

第五次冒泡结束：7，13，22，38，44，5F，D8

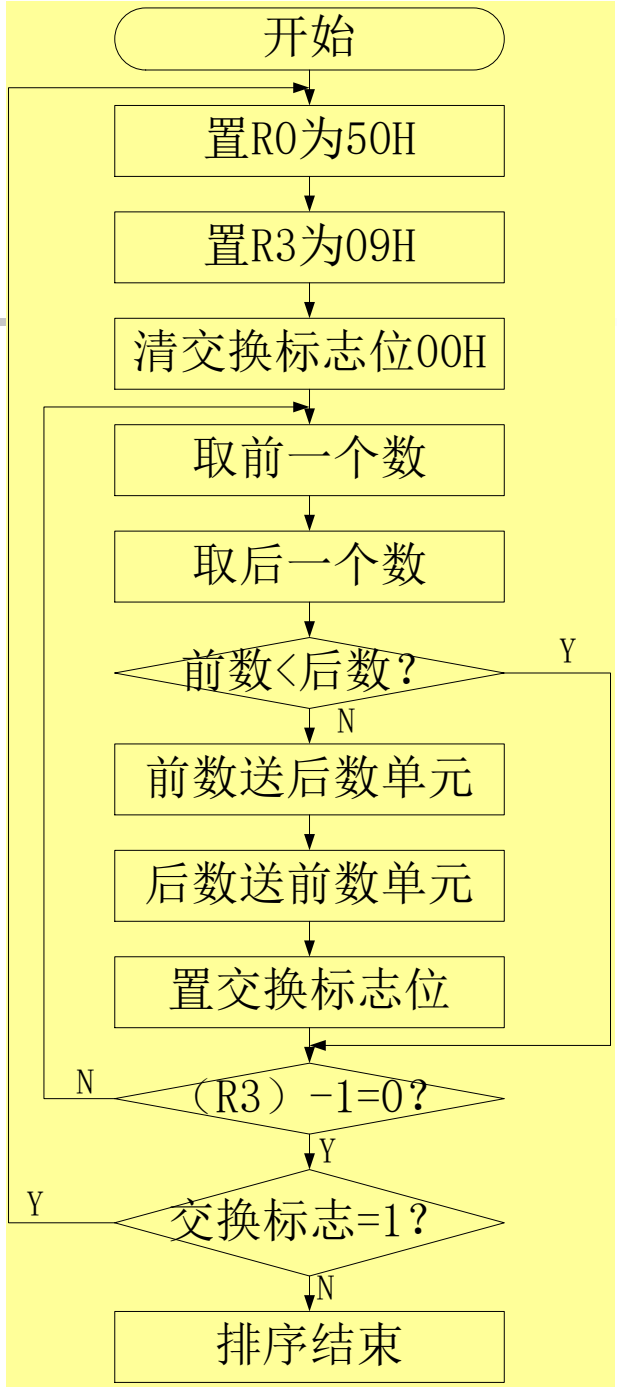


说明：

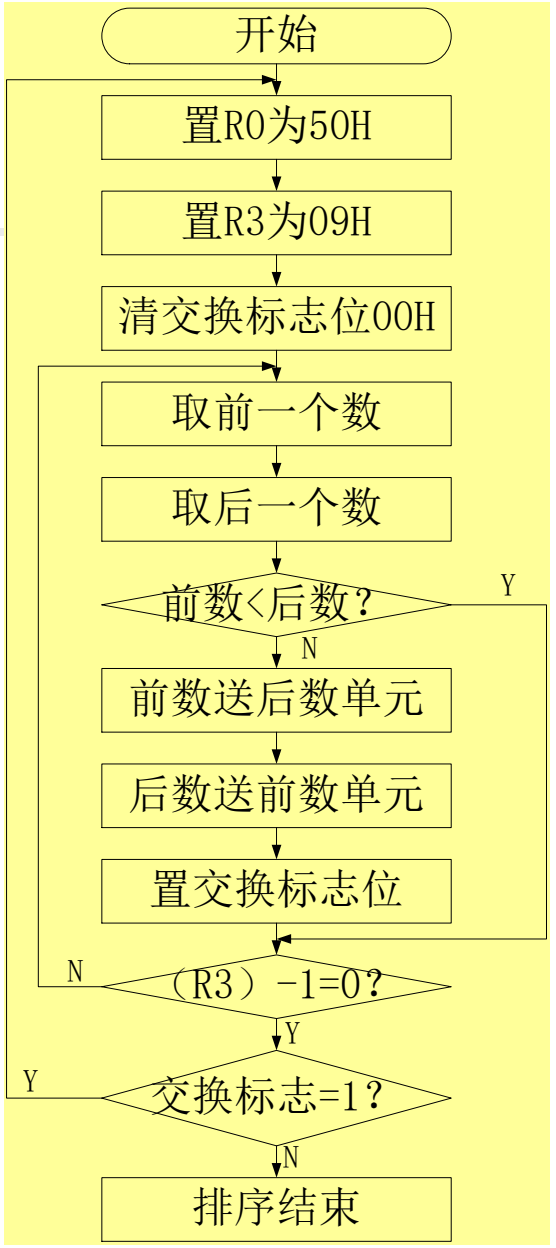
⑴ 每次冒泡都从向后排定了一个大数（升序），每次冒泡所需进行 的比较次数都递减，例如有n个数排序，则第一次冒泡需比较（n-1）次，第二次冒泡则需（n-2）次，依此类推，但实际编程中为了简化程序，往往把各次比较次数都固定为（n-1）次。

⑵ **对于n个数，理论上说应进行（n-1）次冒泡才能完成排序，**但实际上往往不到（n-1）次就已排好序。判定排序是否完成的最简单方法是每次冒泡中是否有互换发生，如果有数据，说明排序还没完成，否则就表示已排好序，为此控制排序结束常不使用计数方法，而使用设置互换标志的方法，以其状态表示在一次冒泡中有无数据互换进行。

例21. 一组测量数据中，共十个，存放于内部RAM 50H~59H单元中，用**冒泡算法**进行升序排序（最小数放在50H，最大值放在59H），位00H用来指示有无交换发生，1代表有交换



```
        ORG    0000H
PX:     CLR    00        ;设交换过标志
        MOV    R3, #09H;比较次数初值为（n-1）次
        MOV    R0, #50H;10个数的数据区首地址
PX1:    MOV    A, @R0   ;取前一个数
        INC    R0        ;调整指针，指向后一个数
        MOV    R1, A     ;前一个数存R1
        CLR    C         ;
        SUBB   A, @R0    ;Xi - Xi+1 （前数 - 后数）
        MOV    A ,  R1   ;恢复前一个数到A
```

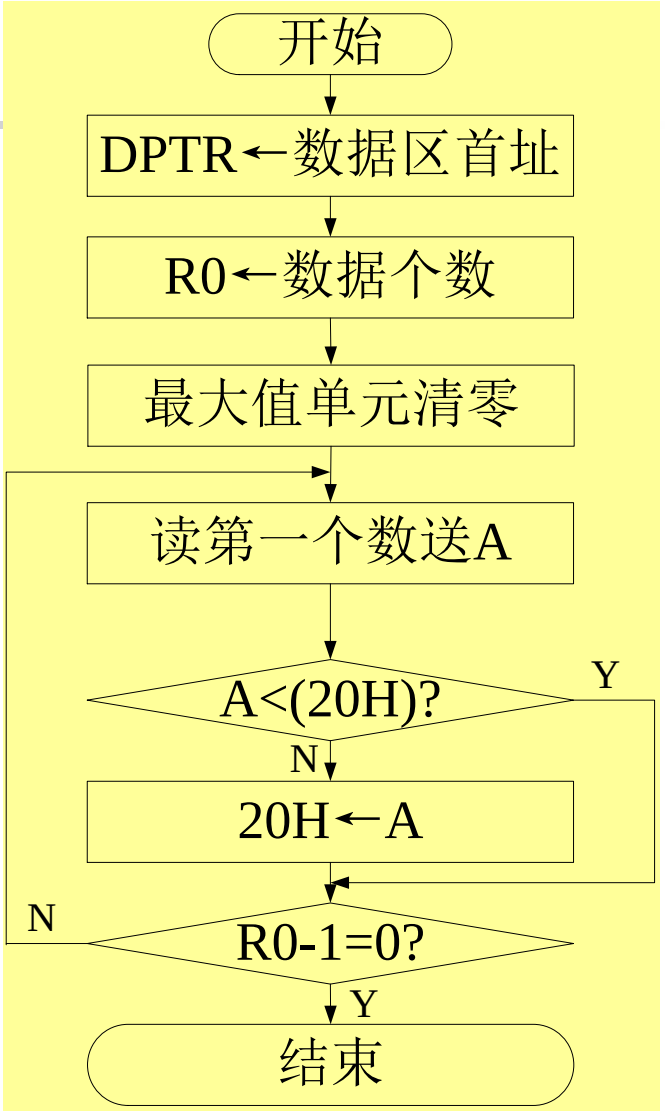




```
        JC      PX2     ; X i <=X i+1则转PX2，不交换
        SETB    00H     ; X i  > =X i+1置交换标志位，00H=1
        XCH     A, @R0  ; X i 、 X i+1交换，A=Xi+1
        DEC     R0      ; 调整指针，指向前数
        XCH     A, @R0  ; 后数送前数位置
        INC     R0      ; 调整指针，指向后数
PX2:    DJNZ    R3, PX1 ; 比较九次
        JB      00H,PX  ; 有交换则再比较一遍
END0:   SJMP    END0
```

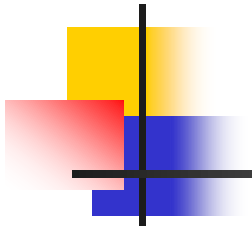
- 查找最大值程序

已知采样值存放在外部RAM1000H至100FH中，试编程找出其中的最大值存入内部RAM区的20H中。



```
        ORG     0000H
        MOV     R0,#10H          ; 采样值数据区长度
        MOV     DPTR,#1000H      ; 采样值存放首址
        MOV     20H,#00H         ; 最大值单元初始值设为最小数
LP:     MOVX    A,@DPTR          ; 取采样值
        CJNE    A,20H,CHK        ; 数值比较
        SJMP    LP1              ; 相等则继续下一次比较
CHK:    JC      LP1              ; A值小，转移
        MOV     20H,A            ; A值大，则送20H
LP1:    INC     DPTR             ; 调整指针，指向下一个
        DJNZ    R0，LP           ; 继续
HERE:   SJMP    HERE             ; 结束
```

注：**20H中始终存放二个数比较后的大值，比较结束后存放的即是最大值。**



➢ 数据搜索程序

- 在数据区中寻找关键字，称为数据搜索。
- 常用的二种方法：**顺序搜索和对分搜索**
- 顺序搜索：

把关键字与数据区中的数据逐个比较，相等者即为找到的关键字。

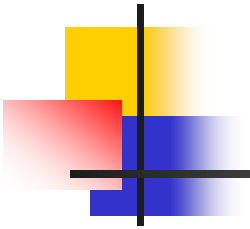
- 对分搜索：

按对分原则进行取数与关键字比较，但前提是数据区中的数已排好序，这样搜索一次后，搜索的数据区范围缩小一半，搜索速度快

例： 已知数据区内有16个数，从内部RAM30H开始存放，要搜索的关键字在20H中，若数据区中搜索到关键字，则在21H中记录关键字在数据区中的序号，若数据区中没有搜索到关键字，则置用户标志F0为1。

采用顺序搜索方法

```
        ORG    0000H
        MOV    R0，#30H        ；数据区首址
        MOV    R1，#16         ；数据区长度
        MOV    20H，#KEY       ；关键字送20H单元
        CLR    F0              ；清用户标志位
        MOV    21H，#01        ；序号置1
LP:     MOV    A，@R0          ；取数
        CJNE   A，20H，LP1
        SJMP   HERE            ; 找到关键字，结束
LP1:    INC    21H             ；序号加1
        INC    R0              ；数据区地址指针加1
        DJNZ   R1，LP；继续
        SETB   F0              ；未搜索到关键字，则置位用户标志
HERE:   SJMP   HERE
```



对分检索：

适合数据量大，而且数据已经排序的场合

4.4 本章小结

- 掌握4种典型的程序结构：

  顺序，分支，循环，子程序

- 掌握几种典型的程序算法：

- **数据排序（冒泡法）、极值查找、搜索编程**
- **多字节无符号数加法/减法**
- **查表**
- **定时/延时程序**

- 能够按照要求编写完整的程序，包括伪指令

第四章作业

- 6，14，15
- 16，18，21